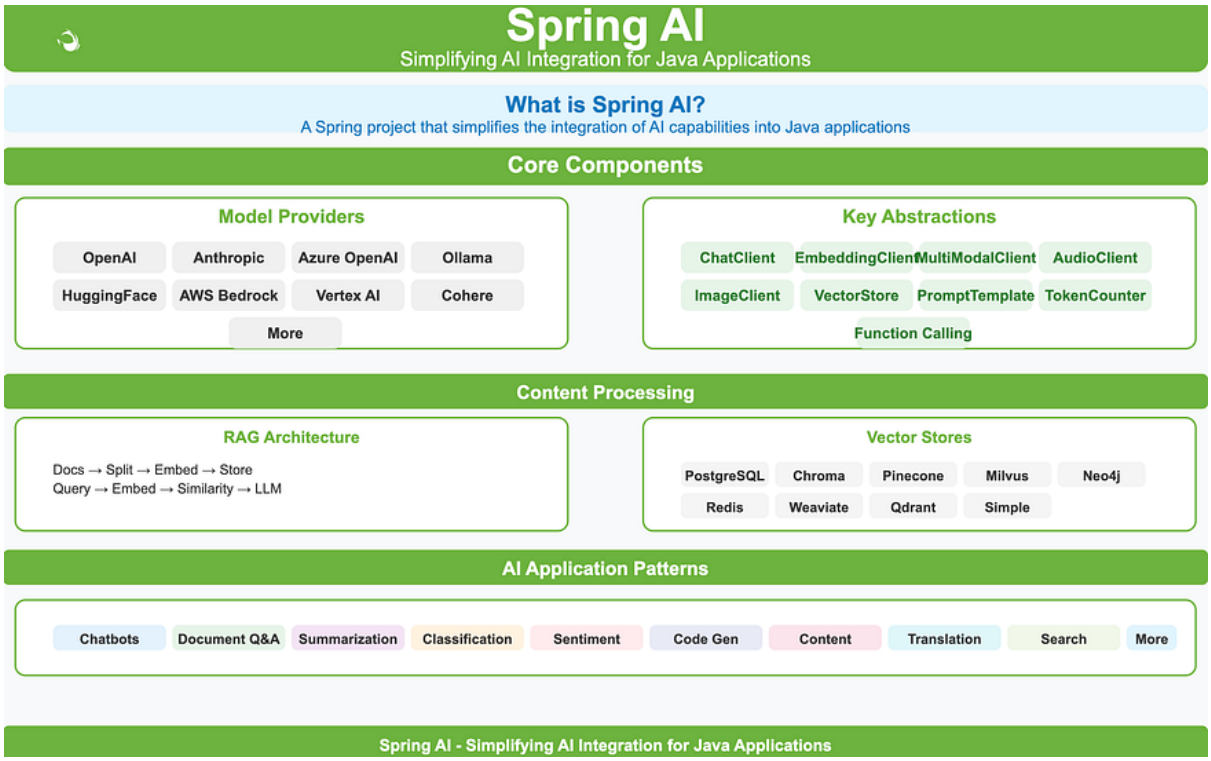
Learning Spring AI

- In the rapidly evolving landscape of artificial intelligence, integrating AI capabilities into applications has become increasingly important for developers.
- However, working with various AI services and models often presents challenges due to inconsistent APIs, complex integration requirements, and the need to understand multiple provider-specific implementations.



- Enter **Spring AI** — a project from the Spring ecosystem designed to simplify AI integration for Java applications.
- If you're familiar with Spring Framework's approach to simplifying enterprise Java development, Spring AI follows the same philosophy by providing consistent abstractions over various AI technologies.

What is Spring AI?

- Spring AI is an extension of the Spring Framework that provides abstractions and utilities for integrating AI capabilities into Java applications.
- Just as Spring simplifies enterprise Java development with its dependency injection and templating patterns, Spring AI simplifies working with large language models (LLMs), embedding models, and other AI technologies.
- The project aims to:

✓ Provide consistent interfaces across different AI service providers

✓ Simplify the integration of AI capabilities into Spring applications

✓ Reduce the learning curve for working with AI technologies

- ✓ Enable developers to switch between AI providers with minimal code changes
- ➢ With Spring AI, you can focus on building your application's business logic rather than dealing with the intricacies of various AI service implementations.

Core Components of Spring AI

Spring AI's architecture consists of two main components: **Model Providers** and **Key Abstractions**. Let's explore each in detail:

Model Providers

Spring AI supports integration with numerous AI service providers, allowing you to choose the best fit for your application's needs:

- **OpenAI**: Access to GPT models like GPT-4o and GPT-3.5-Turbo
- **Anthropic**: Integration with Claude models for natural language processing
- **Azure OpenAI**: Microsoft's managed version of OpenAI's models with enterprise features
- **Ollama**: Local deployment of language models for privacy-sensitive applications
- **HuggingFace**: Access to thousands of open-source models for various AI tasks
- **AWS Bedrock**: Amazon's foundation model service with models from Anthropic, AI21, and others
- **Vertex AI**: Google Cloud's machine learning platform with text, vision, and multimodal models
- **Cohere**: Specialized models for text understanding and generation
- **MistralAI**: Provider of efficient and powerful language models
- **And more**: Spring AI continues to add support for additional providers

The beauty of Spring AI is that you can switch between these providers with minimal code changes. This flexibility allows you to:

1. Compare different models for your specific use case
2. Choose more cost-effective options during development
3. Handle provider-specific outages by quickly switching to alternatives
4. Migrate to different providers as your needs evolve

Key Abstractions

Spring AI provides several key interfaces that abstract away the implementation details of different AI providers:

- **ChatClient**: For conversational AI capabilities, handling message sequences, roles, and streaming responses
- **EmbeddingClient**: For transforming text into vector representations (embeddings) useful for semantic search and document similarity

- **MultiModalClient**: For handling inputs combining text with images or other media types
- **AudioClient**: For speech recognition and audio transcription tasks
- **ImageClient**: For image generation and processing capabilities
- **VectorStore**: For storing and retrieving embeddings, enabling semantic search functionality
- **PromptTemplate**: For creating dynamic prompts with variable substitution
- **TokenCounter**: For estimating token usage to manage costs and stay within model context limits
- **Function Calling**: For enabling AI models to call defined functions and tools

Each of these abstractions provides a consistent programming model regardless of the underlying AI provider implementation.

Content Processing with Spring AI

RAG Architecture

- ➢ One of the most powerful capabilities in Spring AI is its support for **Retrieval-Augmented Generation (RAG)** architectures.
- ➢ RAG combines the power of large language models with the ability to retrieve information from your own data sources.
- ➢ Here's how the RAG workflow typically functions in Spring AI:

1. **Document Processing**:

- Documents from various sources (PDFs, web pages, databases) are loaded using document loaders
- These documents are split into smaller chunks using document splitters
- Each chunk is converted into embeddings (vector representations) using an embedding model
- The embeddings are stored in a vector database along with metadata and the original text

2. Query Processing:

- When a user asks a question, their query is converted into an embedding
- A similarity search finds the most relevant document chunks in the vector store
- These relevant chunks are retrieved and combined with the original query
- The combined information is sent to the LLM, which generates a response based on both the query and retrieved information

- ➢ RAG helps overcome the limitations of LLMs, such as knowledge cutoffs, hallucinations, and lack of access to private data.
- ➢ With Spring AI's abstractions, implementing RAG becomes straightforward by combining the EmbeddingClient, VectorStore, and ChatClient components.

Vector Stores

- A critical component of RAG systems is the vector store, which enables efficient similarity search.
- Spring AI supports multiple vector store implementations:

- **PostgreSQL**: Using the pgvector extension for embedding storage in PostgreSQL databases
- **Chroma**: An open-source embedding database designed specifically for AI applications
- **Pinecone**: A fully managed vector database service built for machine learning applications
- **Milvus**: An open-source vector database for similarity search and AI applications
- **Neo4j**: A graph database with vector search capabilities, useful for connected data
- **Redis**: In-memory data store with vector search via RediSearch module
- **Weaviate**: An open-source vector search engine with multi-modal capabilities
- **Qdrant**: A vector similarity search engine focused on high-performance production use
- **Simple (In-Memory)**: A basic in-memory implementation for testing and development

- The VectorStore abstraction in Spring AI provides consistent methods for storing, retrieving, and searching embeddings across all these implementations.
- This allows you to start simple (e.g., with the in-memory store) and transition to more production-ready solutions (like PostgreSQL or Pinecone) as your application grows.

AI Application Patterns with Spring AI

Conversational Applications

- Building conversational applications is one of the most common use cases for Spring AI.
- The ChatClient abstraction makes it easy to implement chatbots, virtual assistants, and other conversational interfaces.
- Key features for conversational applications include:

  - ✓ **Stateless Messages**: Each message can contain user inputs, system prompts, and roles
  - ✓ **History Management**: Easily track conversation context over multiple interactions
  - ✓ **System Prompts**: Set behavior instructions for the AI model
  - ✓ **User/Assistant Roles**: Define clear roles in the conversation
  - ✓ **Streaming Support**: Process responses as they're generated rather than waiting for complete responses

  Here's a simple example of how a conversation might be implemented:

```
@Service
public class ChatService {
   private final ChatClient chatClient;

   public ChatService(ChatClient chatClient) {
      this.chatClient = chatClient;
   }
```

```
    public String chat(String userMessage, List<ChatMessage> history) {
        // Create a new message with the user's input
        ChatMessage newMessage = new UserMessage(userMessage);

        // Combine with history
        List<ChatMessage> conversation = new ArrayList<>(history);
        conversation.add(newMessage);

        // Create prompt with the full conversation
        Prompt prompt = new Prompt(conversation);

        // Get response from the model
        ChatResponse response = chatClient.call(prompt);

        return response.getResult().getOutput().getContent();
    }
}
```

Advanced Features

- Spring AI offers numerous advanced features for building sophisticated AI applications:
- **Generation Capabilities**
  - ✓ **Text Completions**: Generate coherent text based on provided prompts
  - ✓ **Chat Completion**: Create conversational responses with memory of previous exchanges
  - ✓ **Image Generation**: Create images from text descriptions
  - ✓ **Audio Transcription**: Convert speech to text
  - ✓ **Multimodal Input**: Process combinations of text, images, and other data types
- **Processing Features**
  - ✓ **Embeddings**: Convert text to vector representations for semantic understanding
  - ✓ **RAG Capabilities**: Combine retrieval from your data with AI generation
  - ✓ **Document Loaders**: Import text from various document formats
  - ✓ **Text Splitters**: Break documents into appropriately sized chunks
  - ✓ **Metadata Extraction**: Capture and utilize metadata from documents
- **Tools and Utilities**
  - ✓ **Function Calling**: Allow AI models to call defined functions to extend capabilities
  - ✓ **Tool Execution**: Enable the model to use external tools to enhance responses
  - ✓ **AI Assistants**: Build personalized assistant applications
  - ✓ **Prompt Templates**: Create dynamic prompts with variable substitution
  - ✓ **Token Counting**: Estimate token usage for cost control

Example Applications You Can Build

- With Spring AI, you can build a wide variety of intelligent applications:
- **Chatbots and Conversational Interfaces**
  - ✓ Create interactive chatbots that can assist users, answer questions, and provide information.
  - ✓ The ChatClient abstraction makes it easy to create and maintain conversation history while leveraging the natural language capabilities of LLMs.
- **Document Q&A Systems**
  - ✓ Build systems that can answer questions based on your documents.
  - ✓ By combining document loaders, embeddings, vector stores, and LLMs, you can create applications that understand and respond to queries about your specific content — whether it's technical documentation, knowledge bases, or company policies.
- **Text Summarization**
  - ✓ Create applications that automatically summarize long documents, articles, or reports.
  - ✓ This is particularly useful for processing large volumes of information quickly, such as summarizing meeting transcripts, research papers, or news articles.
- **Classification Systems**
  - ✓ Develop systems that can automatically categorize text, documents, or other content. This can be useful for routing customer inquiries, organizing content, or filtering information.
- **Sentiment Analysis**
  - ✓ Build applications that can analyze the emotional tone of text, useful for understanding customer feedback, social media monitoring, or brand reputation management.
- **Code Generation and Assistance**
  - ✓ Create tools that can assist developers by generating code, explaining code snippets, or suggesting optimizations.
- **Content Creation**
  - ✓ Develop applications for generating marketing copy, product descriptions, articles, or other creative content with AI assistance.
- **Language Translation**
  - ✓ Build translation services that can convert text between multiple languages while preserving context and meaning.
- **Search Enhancement**
  - ✓ Improve search functionality with semantic understanding rather than just keyword matching, providing more relevant results to users.
- **Decision Support Systems**
  - ✓ Create systems that can analyze data and provide recommendations or insights to help users make better decisions.

Getting Started with Spring AI

- To start using Spring AI in your project, add the Spring AI dependency to your Maven or Gradle project.
- For more information, here is the link. https://docs.spring.io/spring-ai/reference/api/chatclient.html

  For Maven:

  ```
  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter</artifactId>
    <version>0.8.0</version>
  </dependency>
  ```

  For Gradle:

  ```
  implementation 'org.springframework.ai:spring-ai-starter:0.8.0'
  ```

  You'll also need to add dependencies for specific AI providers you want to use. For example, to use OpenAI:

  ```
  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
    <version>0.8.0</version>
  </dependency>
  ```

  Then configure your API keys in your application.properties or application.yml file:

  ```
  spring.ai.openai.api-key=your-api-key-here
  ```

- Here's a simple example of creating a Spring Boot application that uses the ChatClient to interact with an AI model:

  ```
  @SpringBootApplication
  @RestController
  public class SpringAiDemoApplication {

  private final ChatClient chatClient;
    public SpringAiDemoApplication(ChatClient chatClient) {
      this.chatClient = chatClient;
    }
    @GetMapping("/chat")
    public String chat(@RequestParam(value = "message") String message) {
      return chatClient.call(new Prompt(message))
            .getResult()
            .getOutput()
            .getContent();
    }
    public static void main(String[] args) {
      SpringApplication.run(SpringAiDemoApplication.class, args);
  ```

```
    }
}
```

Conclusion

- Spring AI brings the simplicity and elegance of the Spring Framework to the world of artificial intelligence integration.
- By providing consistent abstractions over various AI providers and capabilities, it enables Java developers to incorporate AI into their applications without getting lost in provider-specific implementations.